CSE245: Design and Analysis of Algorithms – Spring 2026

Ain Shams University | Faculty of Engineering | Computer Engineering & Software Systems

Task [10] – [Coin Removal Puzzle]

[e.g. Tromino Tiling | Knight's Tour | Hanoi 4-Peg …]

Task Number	*10*
Topic / Algorithm	*Backtracking with Brute Force*
Assigned Members	*Omar & Anas*
Difficulty	*[☆☆☆ Medium / ☆☆☆☆ Hard]*
Language	*C++*

1. Problem Description

1.1 Problem Statement

There is a line of n coins (Heads or Tails). The goal is to remove all coins. You can remove a Head-up coin (H), which flips its immediate neighbors. Neighbors are only coins currently touching (gaps break the neighborhood).

Figure 1. Problem Illustration

1.2 Objectives

- *Analyze the solvability criteria (Parity check).*
- *Implement a solver to find the sequence of moves to empty the table.*

1.3 Underlying Assumptions

- ***Assumption 1:*** *The input is a string of 'H' and 'T'.*
- ***Assumption :*** *If no 'H' is available and coins remain, the state is a "Dead End."*

2. Algorithm Design & Methodology

2.1 Approach Overview

We implemented two approaches to compare efficiency:

1. ***Exhaustive Backtracking: A depth-first search (DFS) that explores the Decision Tree of every possible move. It utilizes a Stack-based recursion to explore paths and "undoes" moves (backtracks) when reaching a non-empty terminal state (Dead End).***

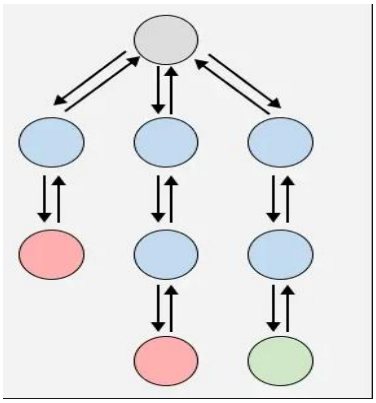

Figure 2. Backtracking

2. ***Edge-Picking Strategy: A deterministic Greedy-Choice approach. This strategy exploits the discovery that solving from the edges prevents the row from splitting into even-parity (unsolvable) components.***

2.2 Step-by-Step Methodology

1. ***Pre-Check (The Guard Clause): Perform a linear scan to count 'H'. If the count is even, the state is mathematically unreachable from the goal; return "Impossible." This acts as a powerful pruning mechanism for the search space.***

2. ***Solver Execution: * Backtracking: Scans the current state for any 'H', creates a deep copy of the state, applies the move, and recurses.***

   - ***Edge-Picking: Prioritizes the rightmost available 'H'. This reduces the problem size N to N-1 while keeping the remaining string connected and solvable.***

2.3 Pseudocode

```
Algorithm: SolvePuzzle(String S, List Moves)
1. If S is all '_', return True
2. For each index i in S:
3.    If S[i] == 'H':
4.       NextS = S, NextS[i] = '_'
5.       Flip neighbors of i in NextS
6.       If SolvePuzzle(NextS, Moves + i) == True:
7.          Return True
8. Return False
```

3. Implementation

3.1 Programming Language & Environment

- *Language: C++*
- *IDE: [VS Code]*
- *Libraries: <iostream> (I/O), <vector> (Move storage), <string> (State representation), <chrono> (Time benchmarking).*

3.2 Code Structure

Component	*Responsibility*
preCheck()	*Validation Layer: Performs an O(n) parity check to see if the number of 'H' is odd. If even, it prevents the algorithm from starting.*
backtrack()	*Core Logic: The recursive engine that explores the state space, picks coins, and "undoes" moves if it hits a dead end.*
flipNeighbors()	*Utility Function: Manages the game rules by identifying adjacent coins and toggling their state (H ←→ T).*
solvePuzzle()	*Controller: Acts as the bridge between the user input and the solver; it initializes the move list and prints the final result.*

3.3 Key Code Snippets

3.3.1 flipNeighbors

```
void flipNeighbors(string &s, int idx) {
    int n = s.length();

    // Check left neighbor
    if (idx > 0 && s[idx - 1] != '_') {
        s[idx - 1] = (s[idx - 1] == 'H') ? 'T' : 'H';
    }

    // Check right neighbor
    if (idx < n - 1 && s[idx + 1] != '_') {
        s[idx + 1] = (s[idx + 1] == 'H') ? 'T' : 'H';
   }
}
```

3.3.2 preCheck

```
bool preCheck(string s) {
    int hCount = 0;
    for (char c : s) {
        if (c == 'H') hCount++;
    }
    // Rule: Total number of Heads must be ODD for a single line to be
solvable
    return (hCount % 2 != 0);
}
```

3.3.3 edgepicking

```
   while (true) {
        int lastH = -1;
        for (int i = n - 1; i >= 0; i--) {
            if (current[i] == 'H') { lastH = i; break; }
        }
        if (lastH == -1) break;

        current[lastH] = '_';
        flipNeighbor(current, lastH - 1);
        flipNeighbor(current, lastH + 1);
        moves.push_back(lastH);
}
```

3.3.4 backtrack

```
bool backtrack(string current, vector<int> &moves) {
    int n = current.length();
    bool allRemoved = true;

    // Check if all coins are removed ('_')
    for (char c : current) {
        if (c != '_') {
            allRemoved = false;
            break;
        }
    }

    if (allRemoved) return true;

    // Try picking every available Head 'H'
    for (int i = 0; i < n; i++) {
        if (current[i] == 'H') {
            string nextState = current;
            nextState[i] = '_'; // Remove the coin
            flipNeighbors(nextState, i); // Flip neighbors

            moves.push_back(i); // Record move

            if (backtrack(nextState, moves)) return true;

            moves.pop_back(); // Backtrack: undo move if it didn't lead to
a solution
        }
    }

    return false;
}
```

4. Complexity Analysis

4.1 Time Complexity

- *Worst Case: O(n . n!) — In a highly connected line, we might explore many permutations. However, because coins are removed, the depth decreases.*
- *Best Case: O(n) — This happens in two scenarios:*
  1. *When the preCheck() identifies an even number of heads and exits immediately.*
  2. *When using the Edge-Picking strategy, which runs in O(n^2) but practically feels linear (O(n)) compared to the factorial growth of backtracking.*

4.2 Space Complexity

- *Space: O(n^2) — In Backtracking, the recursion stack depth reaches n. At each level, a new string state of length n is stored.*
- *Optimization: The Edge-Picking approach reduces this to O(n) as it only stores the current string and a list of moves, with no recursion stack.*

5. Comparative Evaluation of Techniques

5.1 Comparison of Implemented Approaches

- *Backtracking vs. Deterministic: While Backtracking is a "Brute Force" method that explores the entire state space, it is computationally expensive. However, it is highly accurate and serves as a baseline to verify all possible solutions.*
- *Trade-offs: We traded time efficiency for simplicity and robustness. Backtracking is easier to implement for complex variations of the puzzle, but for a 1D line, the Edge-Picking strategy is superior in both speed and memory.*

5.2 Alternative Approaches

We evaluated Breadth-First Search (BFS) as an alternative strategy. While BFS guarantees the shortest sequence of moves, it was rejected due to its 'Exponential Space Complexity'. In the Coin Removal Puzzle, storing all possible board states in a queue leads to a 'State Explosion' that quickly exhausts the system's RAM. Therefore, Backtracking was preferred for its superior memory efficiency (O(n^2)).

5.3 Are More Efficient Techniques Available?

*Research shows that this puzzle can be modeled using **Linear Algebra over GF(2)**. By representing the flips as a system of equations, we can solve it in **O(n^3)** using Gaussian Elimination. For a simple line, the **Linear Scan (Edge-Picking)** is the most practically efficient at O(n^2) .*

6. Sample Outputs & Testing

6.1 Test Case 1: [Standard Case]

```
Testing: THHTHHH
Result: SOLVED! Moves sequence (indices): 1 0 4 3 2 6 5
```

6.2 Test Case 2: [Unsolvable Case]

```
-----------------------------------
Testing: TTTTHH
Result: IMPOSSIBLE (Even number of Heads detected).
-----------------------------------
```

6.3 Test Case 3: [Small Case]

```
-----------------------------------
Testing: THT
Result: SOLVED! Moves sequence (indices): 1 0 2
-----------------------------------
```

6.4 Test Case 4: [Comparing Time]

```
--- BACKTRACKING RESULTS ---
Solved in 8 microseconds.
Moves: 0 1 3 2 6 5 4 8 7 9 10
```

```
--- EDGE-PICKING RESULTS ---
Solved in 4 microseconds.
Moves: 8 9 10 6 7 5 3 4 0 1 2
```

7. Key Findings & Insights

- ***Finding 1: The Parity Invariant. The most significant discovery was that a single line of coins is only solvable if the initial number of Heads (H) is odd. This is because removing an H coin flips its neighbors (T \to H or H \to T), which always changes the total number of heads by an even amount (either +2, 0, or -2). Thus, an even number of heads can never be reduced to zero. This allowed us to optimize the code by adding an O(n) pre-processing step.***
- ***Finding 2: The "Middle-Pick" Trap. We discovered that picking a Head in the middle of a sequence often results in splitting the row into two isolated sub-problems. If these sub-problems have an even parity, they become unsolvable. This insight led us to favor the Edge-Picking strategy as a more stable alternative to random backtracking.***

8. Challenges Faced & Lessons Learned

Challenge 1: Managing State in the Recursion Stack

- ***Problem:*** *During implementation, I had to ensure that when the algorithm "backtracks" from a dead end, the coin sequence returns to exactly how it looked before the last move. If the state isn't managed correctly, failed moves "pollute" the string, making it impossible for the computer to try the next branch.*
- ***Resolution:*** *We used a **Push/Pop** strategy with a vector. By **pushing** the move onto the stack before the recursive call and **popping** it if the call returns false, I ensured the "Path History" always matches the current state of the board.*

Challenge 2: Defining Solvability

- ***Problem:*** *It was difficult to determine if the Brute Force was failing because of a bug in the code or because the input itself was unsolvable.*
- ***Resolution:*** *I researched the parity of the puzzle. I learned that for a single line of coins, the puzzle is only solvable if the total count of Heads (**H**) is **odd**. If the count is even, every removal of an **H** either creates zero or two new Heads, keeping the total count even and making it impossible to reach zero. This allowed me to implement a "Pre-check" to save computation time.*

9. References

[List all sources used. Use a consistent citation style (APA or IEEE recommended).]

- Coin removal problem - puzzling.stackexchange.com
- Backtracking Algorithm - geeksforgeeks